

```
1 !pip -q install yfinance
```

```
1 import os, math, itertools
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import yfinance as yf
7
8 from google.colab import drive
9 drive.mount('/content/drive')
10
11 BASE = '/content/drive/MyDrive/BUSA310/Assignment2'
12 CODE_DIR = f'{BASE}/Colab_Codes'
13 RESULTS_DIR = f'{BASE}/Results'
14 os.makedirs(CODE_DIR, exist_ok=True)
15 os.makedirs(REULTS_DIR, exist_ok=True)
16
17 START = '2025-09-06'
18 END = '2026-09-06'
19
20 # Yahoo Finance uses BTC-USD rather than BTCUSD.
21 tickers_yf = ['SPY', 'TSLA', 'GM', 'BTC-USD', 'AAPL', 'JNJ', 'XOM', 'EFA']
22 display_names = {
23     'SPY': 'SPY',
24     'TSLA': 'TSLA',
25     'GM': 'GM',
26     'BTC-USD': 'BTCUSD',
27     'AAPL': 'AAPL',
28     'JNJ': 'JNJ',
29     'XOM': 'XOM',
30     'EFA': 'EFA'
31 }
```

Mounted at /content/drive

```
1 # Section 1 – Data Collection and Organization
2 end_plus_one = (pd.Timestamp(END) + pd.Timedelta(days=1)).strftime('%Y-%m-%d')
3
4 df = yf.download(
5     tickers_yf,
```

```

6     start=START,
7     end=end_plus_one,
8     auto_adjust=False,
9     group_by='column',
10    progress=False,
11    threads=True
12 )
13
14 print('FIRST 15 ROWS OF RAW DATA')
15 display(df.head(15))
16
17 print('\nRAW DATA INFO')
18 df.info()
19
20 print('\nRAW DATA DESCRIBE')
21 display(df.describe())

```

FIRST 15 ROWS OF RAW DATA

Price Adj Close

Ticker AAPL BTC-USD EFA GM JNJ SPY

Date

2025-09-06	NaN	110224.695312	NaN	NaN	NaN	NaN
2025-09-07	NaN	111167.617188	NaN	NaN	NaN	NaN
2025-09-08	237.004517	112071.429688	89.641426	57.702179	174.204178	641.719543
2025-09-09	233.487518	111530.546875	89.341682	57.513798	173.059982	643.203125
2025-09-10	225.955322	113955.359375	89.322342	56.849533	171.915756	645.062500
2025-09-11	229.183411	115507.539062	90.269913	58.207813	174.566040	650.423157
2025-09-12	233.208542	116101.578125	89.912155	57.999611	174.135712	650.205505
2025-09-13	NaN	115950.507812	NaN	NaN	NaN	NaN
2025-09-14	NaN	115407.656250	NaN	NaN	NaN	NaN

2025-09-15	235.828842	115444.875000	90.511635	58.445763	173.490265	653.667175
2025-09-16	237.273514	116843.187500	90.260239	57.959953	172.570999	652.767090
2025-09-17	238.110428	116468.507812	89.921822	58.227646	173.294678	651.956116
2025-09-18	237.004517	117137.203125	90.269913	58.753113	170.321671	655.002380
2025-09-19	244.596466	115688.859375	89.921822	58.475506	172.306946	658.246521
2025-09-20	NaN	115721.960938	NaN	NaN	NaN	NaN

15 rows x 48 columns

```
RAW DATA INFO
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 366 entries, 2025-09-06 to 2026-09-06
Freq: D
Data columns (total 48 columns):
#      Column                                Non-Null Count  Dtype
---  -
0      (Adj Close, AAPL)                        251 non-null    float64
1      (Adj Close, BTC-USD)                     366 non-null    float64
2      (Adj Close, EFA)                         251 non-null    float64
3      (Adj Close, GM)                          251 non-null    float64
4      (Adj Close, JNJ)                         251 non-null    float64
5      (Adj Close, SPY)                         251 non-null    float64
6      (Adj Close, TSLA)                        251 non-null    float64
7      (Adj Close, XOM)                         251 non-null    float64
8      (Close, AAPL)                            251 non-null    float64
9      (Close, BTC-USD)                         366 non-null    float64
10     (Close, EFA)                             251 non-null    float64
11     (Close, GM)                             251 non-null    float64
12     (Close, JNJ)                             251 non-null    float64
13     (Close, SPY)                             251 non-null    float64
14     (Close, TSLA)                            251 non-null    float64
15     (Close, XOM)                             251 non-null    float64
16     (High, AAPL)                             251 non-null    float64
17     (High, BTC-USD)                          366 non-null    float64
18     (High, EFA)                             251 non-null    float64
19     (High, GM)                              251 non-null    float64
20     (High, JNJ)                              251 non-null    float64
21     (High, SPY)                              251 non-null    float64
22     (High, TSLA)                             251 non-null    float64
```

```

23 (High, XOM)          251 non-null    float64
24 (Low, AAPL)          251 non-null    float64
25 (Low, BTC-USD)       366 non-null    float64
26 (Low, EFA)           251 non-null    float64
27 (Low, GM)            251 non-null    float64
28 (Low, JNJ)           251 non-null    float64
29 (Low, SPY)           251 non-null    float64
30 (Low, TSLA)          251 non-null    float64
31 (Low, XOM)           251 non-null    float64
32 (Open, AAPL)         251 non-null    float64
33 (Open, BTC-USD)      366 non-null    float64
34 (Open, EFA)          251 non-null    float64
35 (Open, GM)           251 non-null    float64
36 (Open, JNJ)          251 non-null    float64
37 (Open, SPY)          251 non-null    float64
38 (Open, TSLA)         251 non-null    float64
39 (Open, XOM)          251 non-null    float64
40 (Volume, AAPL)       251 non-null    float64
41 (Volume, BTC-USD)    366 non-null    int64
42 (Volume, EFA)        251 non-null    float64
43 (Volume, GM)         251 non-null    float64
44 (Volume, JNJ)        251 non-null    float64
45 (Volume, SPY)        251 non-null    float64
46 (Volume, TSLA)       251 non-null    float64
47 (Volume, XOM)        251 non-null    float64

```

```
dtypes: float64(47), int64(1)
```

```
memory usage: 140.1 KB
```

```
RAW DATA DESCRIBE
```

```
Price Adj Close
```

Ticker	AAPL	BTC-USD	EFA	GM	JNJ	S
count	251.000000	366.000000	251.000000	251.000000	251.000000	251.0000
mean	277.709483	82070.202517	98.417189	75.990014	223.038318	700.2668
std	25.305851	17461.445324	5.795221	8.516238	27.838814	39.5921
min	225.955322	58558.859375	88.220078	54.876556	170.321671	630.3460
25%	258.418808	66937.707031	92.513119	72.452797	201.813942	668.2023
50%	271.620667	77575.593750	99.387245	77.689339	227.414093	685.6561
75%	298.282745	91302.384766	103.143948	81.848335	241.000809	739.2176
max	339.786926	124752.531250	108.809998	90.113647	278.429993	777.8800

```
8 rows x 48 columns
```

```

1 # Extract closing prices robustly.
2 if isinstance(df.columns, pd.MultiIndex):
3     lvl0 = df.columns.get_level_values(0)
4     price_field = 'Adj Close' if 'Adj Close' in lvl0 else 'Close'
5     df_close_raw = df[price_field].copy()
6 else:
7     # Fallback if yfinance returns a single-level DataFrame
8     price_field = 'Adj Close' if 'Adj Close' in df.columns else 'Close'
9     df_close_raw = df[[price_field]].copy()
10
11 df_close_raw = df_close_raw.rename(columns=display_names)
12
13 # Count available observations BEFORE aligning calendars.
14 raw_counts = df_close_raw.notna().sum().sort_values(ascending=False)
15 print('Available observations before alignment:')
16 display(raw_counts.to_frame('non_missing_days'))
17
18 # For cross-asset comparisons, use only dates on which every asset has data.
19 # This avoids forward-filling equity prices over weekends and holidays.
20 df_close = df_close_raw.dropna(how='any').copy()
21
22 print(f'\nAligned df_close shape: {df_close.shape}')
23 display(df_close.head())
24
25 print('\ndf_close.dtypes')
26 display(df_close.dtypes)

```

Available observations before alignment:

non_missing_days 	
Ticker	
BTCUSD	366
AAPL	251
EFA	251
GM	251
JNJ	251
SPY	251
TSLA	251
XOM	251

Aligned df_close shape: (251, 8)

Ticker	AAPL	BTCUSD	EFA	GM	JNJ	SPY
Date						
2025-09-08	237.004517	112071.429688	89.641426	57.702179	174.204178	641.719543
2025-09-09	233.487518	111530.546875	89.341682	57.513798	173.059982	643.203125
2025-09-10	225.955322	113955.359375	89.322342	56.849533	171.915756	645.062500
2025-09-11	229.183411	115507.539062	90.269913	58.207813	174.566040	650.423157
2025-09-12	233.208542	116101.578125	89.912155	57.999611	174.135712	650.205505

df_close.dtypes

0

Ticker	
AAPL	float64
BTCUSD	float64
EFA	float64
GM	float64
JNJ	float64
SPY	float64

```

1 # Section 2 – Descriptive Statistics and Risk Metrics
2
3 stats_table = pd.DataFrame({
4     'Min': df_close.min(),
5     'Max': df_close.max(),
6     'Mean': df_close.mean(),
7     'Std': df_close.std()
8 }).round(2)
9

```

```

10 print('PRICE DESCRIPTIVE STATISTICS')
11 display(stats_table)
12
13 # Annual return using the assignment formula
14 annual_return = (df_close.iloc[-1] - df_close.iloc[0]) / df_close
15 annual_return_pct = (annual_return * 100).round(2)
16
17 print('ANNUAL RETURN (%)')
18 display(annual_return_pct.sort_values(ascending=False).to_frame('')
19
20 # Daily returns
21 daily_returns = df_close.pct_change(fill_method=None).dropna()
22
23 # Sharpe
24 rf_daily = 0.045 / 252
25 sharpe = ((daily_returns.mean() - rf_daily) / daily_returns.std())
26 sharpe = sharpe.sort_values(ascending=False)
27
28 print('ANNUALIZED SHARPE RATIO')
29 display(sharpe.round(3).to_frame('Sharpe'))
30
31 # 30-day rolling std of DAILY returns, shown as %
32 rolling_30 = daily_returns.rolling(30).std() * 100
33 print('LAST 10 ROWS: 30-DAY ROLLING DAILY-RETURN STD (%)')
34 display(rolling_30.tail(10).round(3))
35
36 # Required price-level correlation
37 price_corr = df_close.corr().round(2)
38 print('PRICE-LEVEL CORRELATION MATRIX')
39 display(price_corr)

```

PRICE DESCRIPTIVE STATISTICS

	Min	Max	Mean	Std	
Ticker					
AAPL	225.96	339.79	277.71	25.31	
BTCUSD	58558.86	124752.53	82202.36	17614.71	
EFA	88.22	108.81	98.42	5.80	
GM	54.88	90.11	75.99	8.52	
JNJ	170.32	278.43	223.04	27.84	
SPY	630.35	777.88	700.27	39.59	

TSLA	298.32	489.88	405.59	39.95
XOM	106.76	169.22	136.55	18.89

ANNUAL RETURN (%)

Annual Return %

Ticker	
JNJ	57.99
GM	52.09
XOM	49.38
AAPL	35.01
EFA	20.87
SPY	20.02
TSLA	2.22
BTCUSD	-28.91

ANNUALIZED SHARPE RATIO

Sharpe

Ticker	
JNJ	2.277
XOM	1.529
GM	1.272
AAPL	1.152
SPY	1.147
EFA	1.010
TSLA	0.190
BTCUSD	-0.664

LAST 10 ROWS: 30-DAY ROLLING DAILY-RETURN STD (%)

Ticker	AAPL	BTCUSD	EFA	GM	JNJ	SPY	TSLA	XOM
Date								
2026-08-24	2.042	2.428	0.825	2.101	1.465	0.778	3.562	1.406
2026-08-25	2.038	2.344	0.826	2.126	1.429	0.777	3.562	1.480

2026-08-26	1.904	2.336	0.829	2.123	1.344	0.775	3.566	1.514
2026-08-27	1.874	2.306	0.813	2.125	1.376	0.774	3.606	1.531
2026-08-28	1.902	2.407	0.809	2.074	1.369	0.749	3.589	1.526
2026-08-31	1.869	2.393	0.796	2.069	1.337	0.752	3.698	1.589
2026-09-01	1.932	2.410	0.779	1.901	1.373	0.753	3.714	1.588
2026-09-02	1.929	2.404	0.783	1.824	1.354	0.755	3.708	1.566
2026-09-03	1.921	2.509	0.758	1.849	1.347	0.732	2.674	1.565
2026-09-04	1.863	2.526	0.755	1.807	1.347	0.739	2.889	1.600

PRICE-LEVEL CORRELATION MATRIX

Ticker	AAPL	BTCUSD	EFA	GM	JNJ	SPY	TSLA	XOM	
Ticker									
AAPL	1.00	-0.57	0.74	0.61	0.66	0.89	-0.39	0.49	
BTCUSD	-0.57	1.00	-0.78	-0.75	-0.88	-0.59	0.53	-0.83	
EFA	0.74	-0.78	1.00	0.79	0.89	0.88	-0.55	0.79	
GM	0.61	-0.75	0.79	1.00	0.75	0.66	-0.32	0.64	
JNJ	0.66	-0.88	0.89	0.75	1.00	0.71	-0.66	0.88	
SPY	0.89	-0.59	0.88	0.66	0.71	1.00	-0.42	0.57	
TSLA	-0.39	0.53	-0.55	-0.32	-0.66	-0.42	1.00	-0.67	
XOM	0.49	-0.83	0.79	0.64	0.88	0.57	-0.67	1.00	

Next steps:

[New interactive sheet](#)[New interactive sheet](#)

```

1 # Supporting metrics used for interpretation
2 ann_vol = daily_returns.std() * np.sqrt(252)
3 return_corr = daily_returns.corr()
4
5 # Utility: unique correlation pairs
6 def unique_corr_pairs(corr):
7     pairs = []
8     cols = corr.columns
9     for i in range(len(cols)):
10         for j in range(i+1, len(cols)):

```

```

11         pairs.append((cols[i], cols[j], corr.iloc[i,j]))
12     return pd.DataFrame(pairs, columns=['Ticker1','Ticker2','Corr']
13
14 price_pairs = unique_corr_pairs(df_close.corr())
15 return_pairs = unique_corr_pairs(return_corr)
16
17 highest_price_pair = price_pairs.loc[price_pairs['Correlation'].idxmax()]
18 lowest_price_pair = price_pairs.loc[price_pairs['Correlation'].idxmin()]
19 lowest_return_pair = return_pairs.loc[return_pairs['Correlation'].idxmin()]
20
21 # Peak rolling volatility date for each ticker
22 peak_roll = {}
23 for t in rolling_30.columns:
24     s = rolling_30[t].dropna()
25     peak_roll[t] = (s.idxmax(), s.max())
26
27 print('ANNUALIZED RETURN VOLATILITY (%) – SUPPORTING METRIC')
28 display((ann_vol*100).round(2).sort_values().to_frame('Annualized
29
30 print('\nHighest price-level correlation pair:')
31 print(highest_price_pair.to_dict())
32 print('Lowest price-level correlation pair:')
33 print(lowest_price_pair.to_dict())
34 print('Lowest RETURN-correlation pair (better diversification evic
35 print(lowest_return_pair.to_dict())
36
37 print('\nPeak 30-day rolling daily-return volatility by ticker:')
38 for t,(d,v) in peak_roll.items():
39     print(f'{t}: {d.date()} – {v:.2f}% daily std')

```

ANNUALIZED RETURN VOLATILITY (%) – SUPPORTING METRIC

Annualized return vol % 

Ticker	
SPY	12.83
EFA	15.67
JNJ	19.08
AAPL	25.12
XOM	25.69
GM	34.19
BTCUSD	43.96

TSLA

47.47

Highest price-level correlation pair:

`{'Ticker1': 'AAPL', 'Ticker2': 'SPY', 'Correlation': 0.8942691008582608}`

Lowest price-level correlation pair:

`{'Ticker1': 'BTCUSD', 'Ticker2': 'JNJ', 'Correlation': -0.8834096972336}`

Lowest RETURN-correlation pair (better diversification evidence):

`{'Ticker1': 'SPY', 'Ticker2': 'XOM', 'Correlation': -0.2768259609833507}`

Peak 30-day rolling daily-return volatility by ticker:

AAPL: 2026-08-04 – 2.56% daily std

BTCUSD: 2026-03-16 – 4.73% daily std

EFA: 2026-04-08 – 1.70% daily std

GM: 2025-11-07 – 3.21% daily std

JNJ: 2026-07-30 – 1.94% daily std

SPY: 2026-04-14 – 1.15% daily std

TSLA: 2026-07-23 – 4.28% daily std

XOM: 2026-06-16 – 2.10% daily std

```

1 ranking = annual_return_pct.sort_values(ascending=False)
2 best_ret_ticker = ranking.index[0]
3 best_ret = ranking.iloc[0]
4 value_1000 = 1000 * (1 + best_ret/100)
5
6 best_sharpe_ticker = sharpe.index[0]
7 best_sharpe = sharpe.iloc[0]
8
9 low_price_std_ticker = stats_table['Std'].idxmin()
10 low_price_std = stats_table.loc[low_price_std_ticker, 'Std']
11
12 print('Return ranking:', ' > '.join([f'{t} ({ranking[t]:.2f}%)' for t in ranking.index])
13 print(f'Best return: {best_ret_ticker} at {best_ret:.2f}%')
14 print(f'$1,000 × (1 + {best_ret:.4f}) = ${value_1000:,.2f}.\n')
15
16 print(f'Highest Sharpe: {best_sharpe_ticker} ({best_sharpe:.3f}).')
17 print(f'Highest annual return: {best_ret_ticker} ({best_ret:.2f}%')
18 if best_sharpe_ticker == best_ret_ticker:
19     print('They are the same asset in this sample.')
20 else:
21     print('They are different. This shows that the asset with the
22         'For comparing investments with different risk, Sharpe :
23
24 # Choose the ticker with the single highest peak 30-day vol
25 q7_ticker = max(peak_roll, key=lambda k: peak_roll[k][1])
26 q7_date, q7_vol = peak_roll[q7_ticker]
```

```

27 print(f'Use {q7_ticker}: peak 30-day daily-return volatility = {q7_volatility}')
28
29 print(f'Highest price-level correlation: {highest_price_pair.Ticker1} - {highest_price_pair.Ticker2} = {highest_price_pair.correlation}')
30 print(f'Lowest price-level correlation: {lowest_price_pair.Ticker1} - {lowest_price_pair.Ticker2} = {lowest_price_pair.correlation}')
31 print(f'For diversification, I would focus more on return correlations; the lowest return correlation is between {lowest_return_pair.Ticker1} and {lowest_return_pair.Ticker2} with a correlation of {lowest_return_pair.correlation}')
32     f'{lowest_return_pair.Ticker1}-{lowest_return_pair.Ticker2} = {lowest_return_pair.correlation}')
33 )
34
35 print(f'Lowest closing-price standard deviation: {low_price_std_ticker} = {low_price_std_deviation}')
36 print(f'Its annual return = {annual_return_pct[low_price_std_ticker]} and Sharpe = {low_price_std_sharpe}')
37 alt = best_sharpe_ticker
38 print(f'Compare with {alt}: annual return = {annual_return_pct[alt]} and Sharpe = {alt_sharpe}')
39     f'annualized return volatility = {ann_vol[alt]*100:.2f}%.'')

```

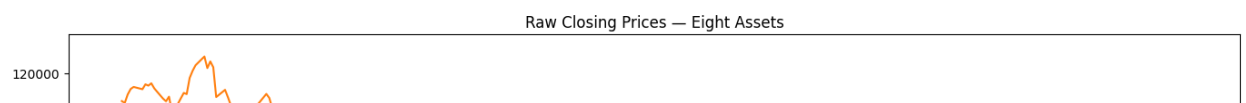
Return ranking: JNJ (57.99%) > GM (52.09%) > XOM (49.38%) > AAPL (35.09%)
 Best return: JNJ at 57.99%.
 $\$1,000 \times (1 + 57.99\%) = \$1,579.90$.

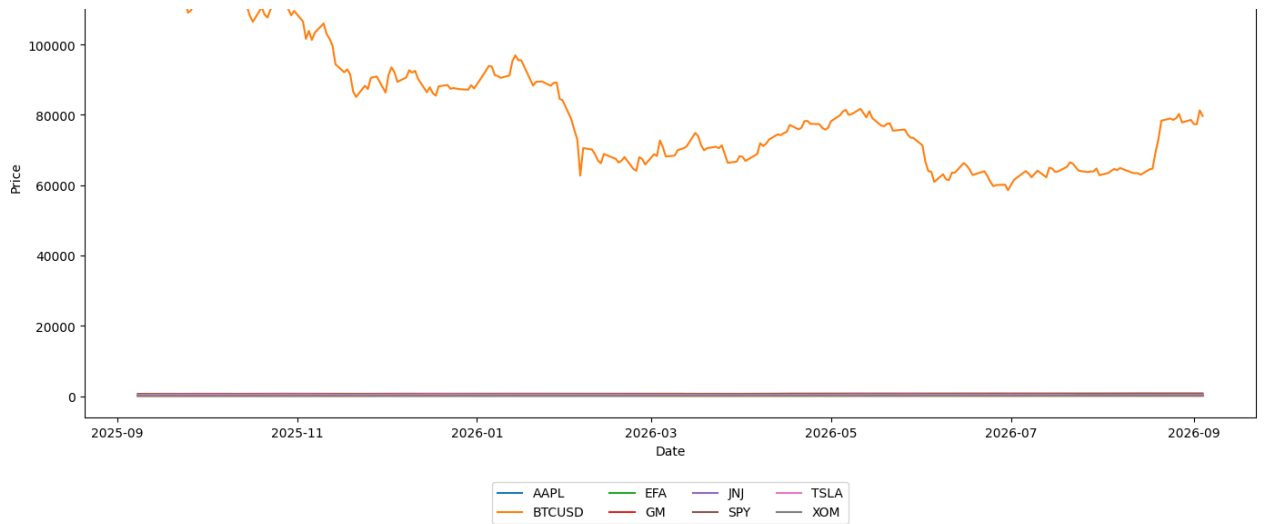
Highest Sharpe: JNJ (2.277).
 Highest annual return: JNJ (57.99%).
 They are the same asset in this sample.
 Use BTCUSD: peak 30-day daily-return volatility = 4.73% around 2026-03-01.
 Highest price-level correlation: AAPL-SPY = 0.89.
 Lowest price-level correlation: BTCUSD-JNJ = -0.88.
 For diversification, I would focus more on return correlations; the lowest return correlation is between BTCUSD and JNJ with a correlation of -0.88.
 Lowest closing-price standard deviation: EFA = 5.80.
 Its annual return = 20.87% and Sharpe = 1.010.
 Compare with JNJ: annual return = 57.99%, Sharpe = 2.277, annualized return volatility = 10.00%.

```

1 # Section 3 – Visualization
2
3 # Part A: Raw price chart
4 plt.figure(figsize=(14,7))
5 for c in df_close.columns:
6     plt.plot(df_close.index, df_close[c], label=c)
7 plt.title('Raw Closing Prices – Eight Assets')
8 plt.xlabel('Date')
9 plt.ylabel('Price')
10 plt.legend(loc='upper center', bbox_to_anchor=(0.5,-0.12), ncol=4)
11 plt.tight_layout()
12 plt.savefig(f'{RESULTS_DIR}/raw_prices.png', dpi=200, bbox_inches='tight')
13 plt.show()

```





```

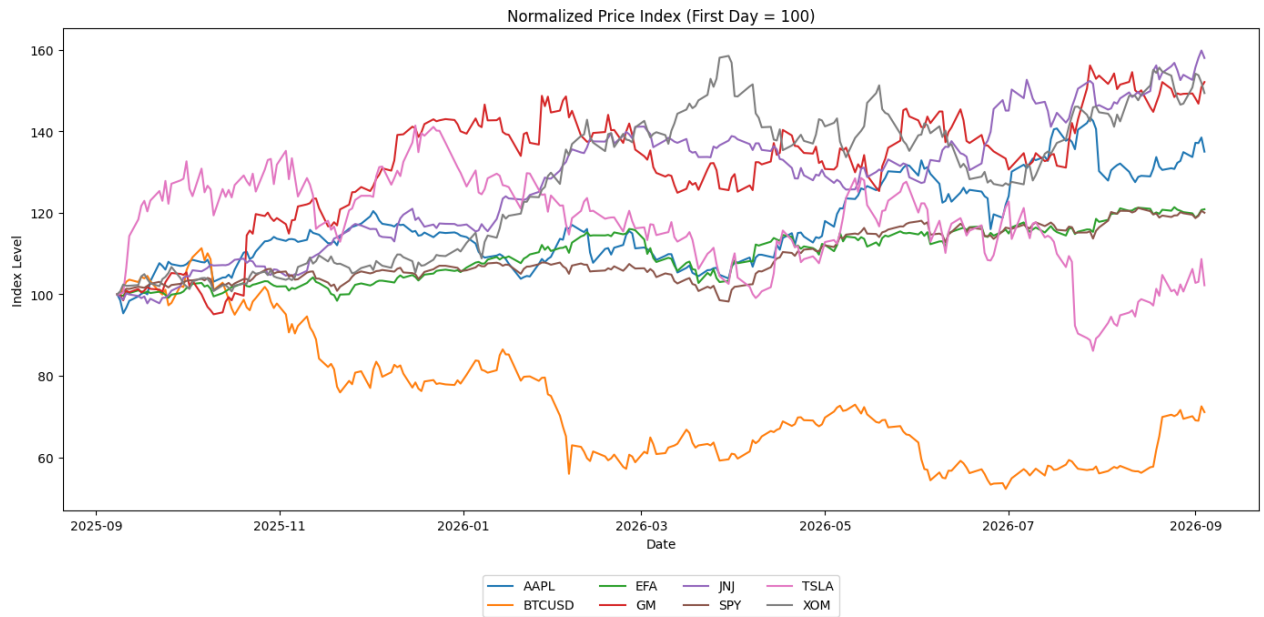
1 # Normalized to 100
2 normalized_df_close = df_close.div(df_close.iloc[0]).mul(100)
3
4 plt.figure(figsize=(14,7))
5 for c in normalized_df_close.columns:
6     plt.plot(normalized_df_close.index, normalized_df_close[c], label=c)
7 plt.title('Normalized Price Index (First Day = 100)')
8 plt.xlabel('Date')
9 plt.ylabel('Index Level')
10 plt.legend(loc='upper center', bbox_to_anchor=(0.5,-0.12), ncol=4)
11 plt.tight_layout()

```

```

12 plt.savefig(f'{RESULTS_DIR}/normalized_prices.png', dpi=200, bbox_
13 plt.show()

```



```

1 pairs = []
2 for a,b in itertools.combinations(normalized_df_close.columns,2):
3     mean_abs_gap = (normalized_df_close[a]-normalized_df_close[b]
4     pairs.append((a,b,mean_abs_gap))
5 closest_pair = min(pairs, key=lambda x:x[2])
6

```

```

7 final_norm = normalized_df_close.iloc[-1]
8 outperformers = final_norm[final_norm > final_norm['SPY']].sort_v
9
10 # Find a 63-trading-day (~3 month) window ending when cross-section
11 spread = normalized_df_close.max(axis=1) - normalized_df_close.min
12 spread_end = spread.idxmax()
13 spread_start = normalized_df_close.index[max(0, normalized_df_clo
14 best_on_spread_end = normalized_df_close.loc[spread_end].idxmax()
15 worst_on_spread_end = normalized_df_close.loc[spread_end].idxmin(
16
17 print(f'The hardest pair to distinguish is {closest_pair[0]} and
18
19 print('Final normalized levels:')
20 display(final_norm.sort_values(ascending=False).round(2).to_frame
21 print('Outperformed SPY:', list(outperformers.index))
22 print(f'Largest cross-sectional spread occurs around {spread_end.
23 print(f'At the spread end, best = {best_on_spread_end}; worst = {

```

The hardest pair to distinguish is EFA and SPY, with the smallest average

Final normalized levels:

Final normalized index 

Ticker

JNJ	157.99
GM	152.09
XOM	149.38
AAPL	135.01
EFA	120.87
SPY	120.02
TSLA	102.22
BTCUSD	71.09

Outperformed SPY: ['JNJ', 'GM', 'XOM', 'AAPL', 'EFA']

Largest cross-sectional spread occurs around 2026-07-28; inspect roughl

At the spread end, best = GM; worst = BTCUSD; spread = 99.18 normalized

```

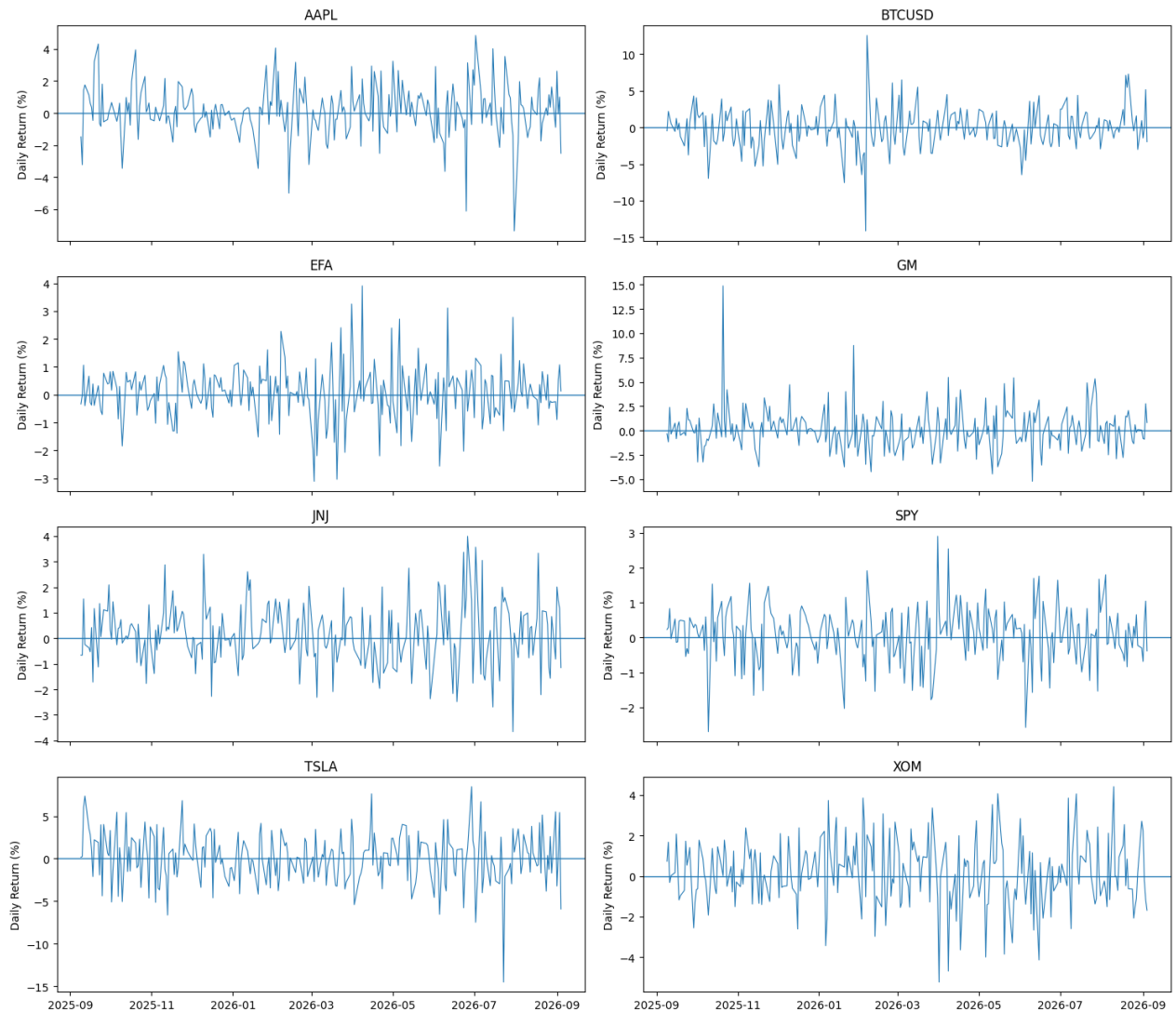
1 # Part B: Daily return subplots
2 returns_pct = daily_returns * 100
3

```

```

4 fig, axes = plt.subplots(4,2, figsize=(15,13), sharex=True)
5 axes = axes.ravel()
6 for ax,t in zip(axes, returns_pct.columns):
7     ax.plot(returns_pct.index, returns_pct[t], linewidth=0.8)
8     ax.axhline(0, linewidth=1)
9     ax.set_title(t)
10    ax.set_ylabel('Daily Return (%)')
11 plt.tight_layout()
12 plt.savefig(f'{RESULTS_DIR}/daily_return_subplots.png', dpi=200, l
13 plt.show()

```





```

1 stacked_abs = returns_pct.abs().stack()
2 largest_idx = stacked_abs.idxmax()
3 largest_date, largest_ticker = largest_idx
4 largest_move = returns_pct.loc[largest_date, largest_ticker]
5
6 # Same-direction co-movement:
7 # Find date maximizing count of >2% positive moves or <-2% negative
8 pos_count = (returns_pct > 2).sum(axis=1)
9 neg_count = (returns_pct < -2).sum(axis=1)
10 co_date = pd.concat([pos_count.rename('pos'), neg_count.rename('neg')])
11 direction = 'positive' if pos_count.loc[co_date] >= neg_count.loc[co_date]
12 count_same = max(pos_count.loc[co_date], neg_count.loc[co_date])
13
14 print(f'Largest single daily move: {largest_ticker} on {largest_date}, {largest_move.pct()}%')
15 print(f'A strong same-direction co-movement date is {co_date.date()} with {count_same} assets')

```

Largest single daily move: GM on 2025-10-21, 14.86%.

A strong same-direction co-movement date is 2026-03-31: 6 assets had moves

```

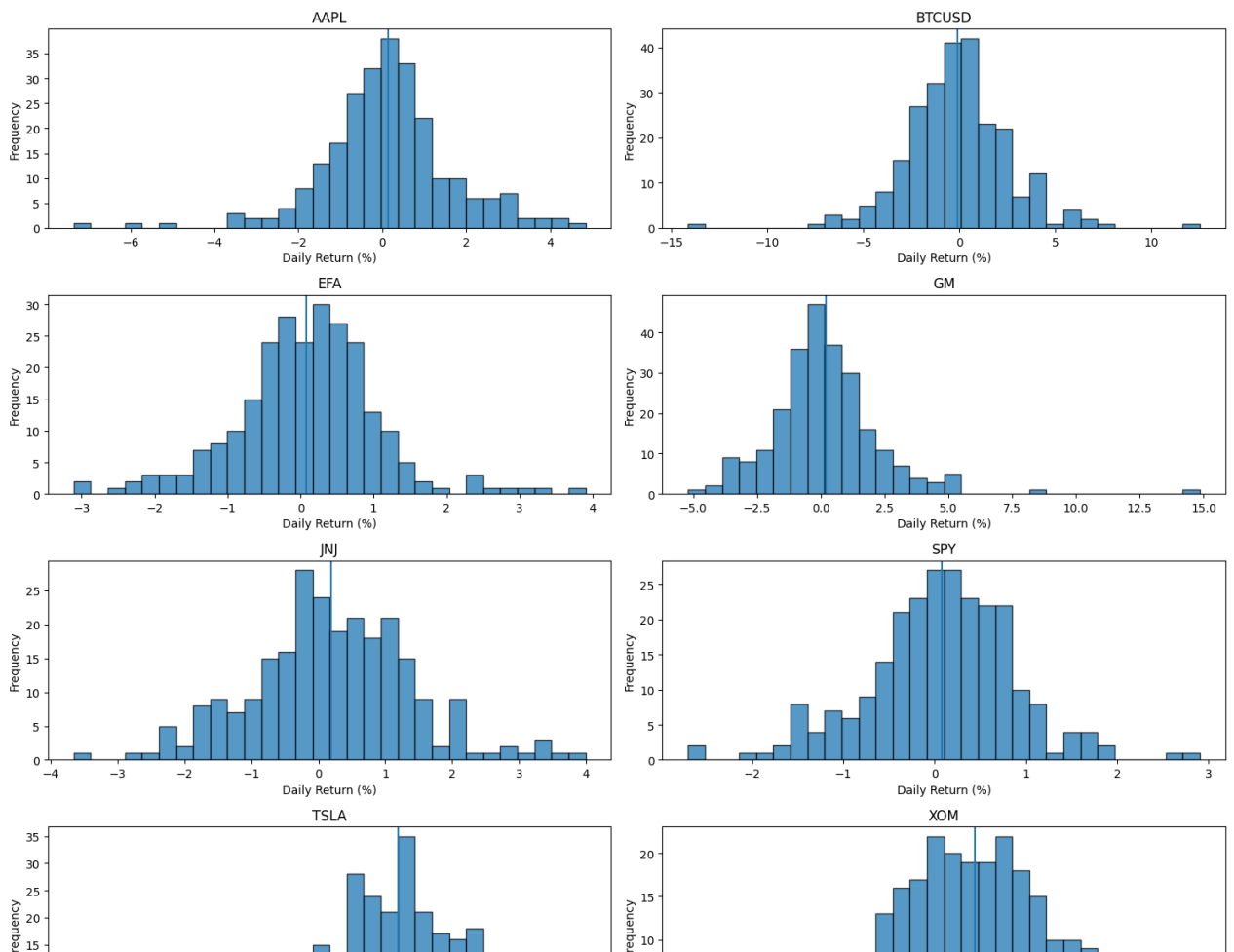
1 # Part C: Histograms
2 fig, axes = plt.subplots(4,2, figsize=(15,13))
3 axes = axes.ravel()
4 for ax,t in zip(axes, returns_pct.columns):
5     ax.hist(returns_pct[t].dropna(), bins=30, edgecolor='black', color='blue')
6     ax.axvline(returns_pct[t].mean(), linewidth=1.5)
7     ax.set_title(t)
8     ax.set_xlabel('Daily Return (%)')
9     ax.set_ylabel('Frequency')
10 plt.tight_layout()
11 plt.savefig(f'{RESULTS_DIR}/return_histograms.png', dpi=200, bbox_inches='tight')
12 plt.show()
13
14 # Shape metrics
15 skewness = daily_returns.skew()
16 excess_kurt = daily_returns.kurt() # pandas returns excess kurtosis
17
18 shape_table = pd.DataFrame({
19     'Skewness': skewness,
20     'Excess Kurtosis': excess_kurt
21 })
22
23 def shape_label(sk, ku):
24     if ku > 1:

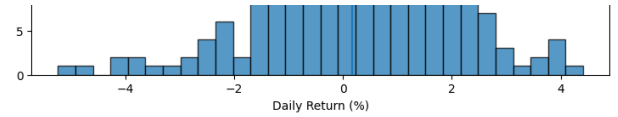
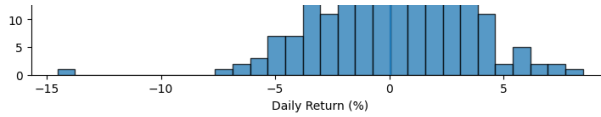
```

```

25     base = 'heavy-tailed'
26     elif sk > 0.5:
27         base = 'right-skewed'
28     elif sk < -0.5:
29         base = 'left-skewed'
30     else:
31         base = 'roughly symmetric'
32     return base
33
34 shape_table['Shape'] = [shape_label(shape_table.loc[t, 'Skewness'],
35 shape_table['Normality score'] = shape_table['Skewness'].abs() +
36 shape_table = shape_table.sort_values('Normality score')
37
38 display(shape_table.round(3))
39
40 most_heavy = excess_kurt.idxmax()
41
42 print('Most normal-looking to least (using |skew| + |excess kurtosis|):')
43 print(' > '.join(shape_table.index))
44 print(f'Most heavy-tailed by excess kurtosis: {most_heavy}, excess kurtosis: {excess_kurt[most_heavy]}')

```





Skewness **Excess Kurtosis** **Shape** **Normality score**

Ticker

JNJ	0.130	0.594	roughly symmetric	0.724
XOM	-0.180	0.578	roughly symmetric	0.757
SPY	-0.209	1.198	heavy-tailed	1.407
EFA	0.136	1.867	heavy-tailed	2.002
TSLA	-0.325	1.740	heavy-tailed	2.065
AAPL	-0.449	3.045	heavy-tailed	3.493
BTCUSD	-0.051	3.860	heavy-tailed	3.911
GM	1.608	8.866	heavy-tailed	10.475

Most normal-looking to least (using $|skew| + |excess\ kurtosis|$):

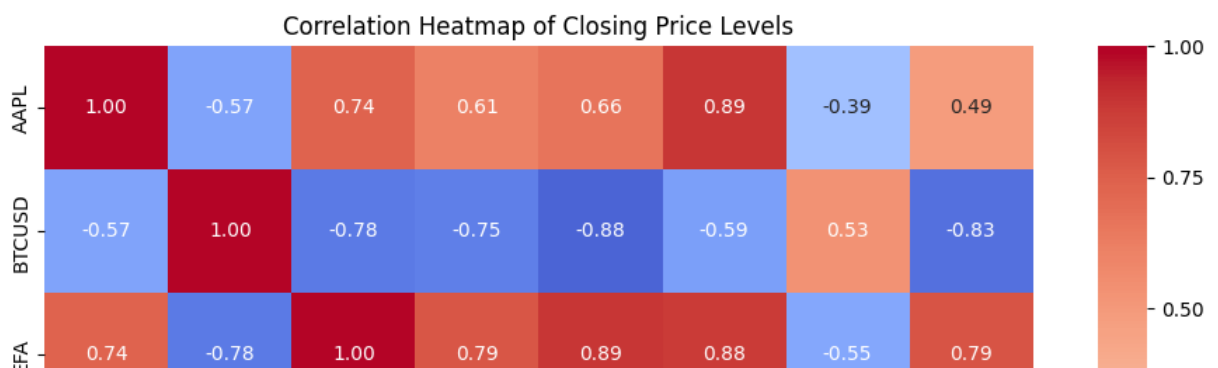
JNJ > XOM > SPY > EFA > TSLA > AAPL > BTCUSD > GM

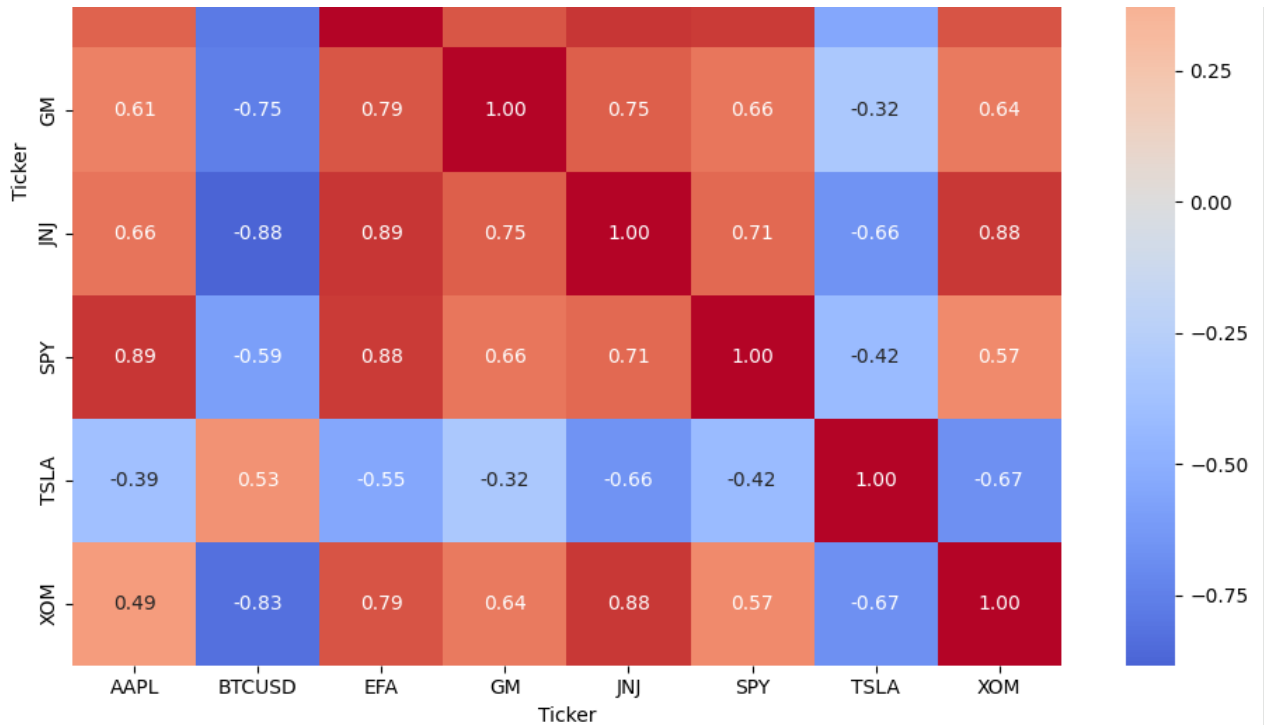
Most heavy-tailed by excess kurtosis: GM, excess kurtosis = 8.87.

```

1 # Part D: Required PRICE-LEVEL correlation heatmap
2 plt.figure(figsize=(10,8))
3 sns.heatmap(df_close.corr(), annot=True, fmt='.2f', cmap='coolwarm')
4 plt.title('Correlation Heatmap of Closing Price Levels')
5 plt.tight_layout()
6 plt.savefig(f'{RESULTS_DIR}/price_correlation_heatmap.png', dpi=200)
7 plt.show()
8
9 avg_corr = df_close.corr().apply(lambda s: (s.sum()-1)/(len(s)-1))
10 most_independent = avg_corr.idxmin()
11
12 # strongest 3-asset cluster: maximize mean pairwise corr
13 best_cluster = None
14 best_cluster_corr = -999
15 for combo in itertools.combinations(df_close.columns,3):
16     sub = df_close[list(combo)].corr()
17     vals = [sub.iloc[0,1], sub.iloc[0,2], sub.iloc[1,2]]
18     m = np.mean(vals)
19     if m > best_cluster_corr:
20         best_cluster_corr = m
21         best_cluster = combo
22
23 print(f'Strongest 3-asset cluster by average price-level correlation: {best_cluster}')
24 print(f'Most independent by lowest average correlation with the rest: {most_independent}')

```





Strongest 3-asset cluster by average price-level correlation: ('EFA', 'JNJ', 'SPY')
 Most independent by lowest average correlation with the rest: BTCUSD, and TSLA

```

1 # Section 4 – Portfolio Construction
2
3 # Client: David, age 61, retirement in 6 years, $40,000 investable
4 # The allocation below deliberately emphasizes diversified/defensive assets
5
6 allocation = pd.Series({
7     'SPY': 45,
8     'TSLA': 0,
9     'GM': 0,

```

```

10     'BTCUSD': 0,
11     'AAPL': 5,
12     'JNJ': 20,
13     'XOM': 15,
14     'EFA': 15
15 }, dtype=float)
16 assert abs(allocation.sum()-100) < 1e-9
17 display(allocation.to_frame('Allocation %'))
18 dollars = allocation/100 * 40000
19 display(dollars.to_frame('Dollar Allocation'))
20
21 print('SPY 45%, AAPL 5%, JNJ 20%, XOM 15%, EFA 15%, TSLA 0%, GM 0%')
22 print('The portfolio emphasizes broad-market and defensive diversification')
23     'GM is excluded to avoid adding another concentrated cyclical stock'

```

Allocation % 	
SPY	45.0
TSLA	0.0
GM	0.0
BTCUSD	0.0
AAPL	5.0
JNJ	20.0
XOM	15.0
EFA	15.0

Dollar Allocation	
SPY	18000.0
TSLA	0.0
GM	0.0
BTCUSD	0.0
AAPL	2000.0
JNJ	8000.0
XOM	6000.0
EFA	6000.0

SPY 45%, AAPL 5%, JNJ 20%, XOM 15%, EFA 15%, TSLA 0%, GM 0%, BTCUSD 0%.

The portfolio emphasizes broad-market and defensive diversification. TS

```
1 print('QUANTITATIVE ANCHORS')
2 for t,w in allocation[allocation>0].items():
3     print(f'{t} ({w:.0f}%): annual return {annual_return_pct[t]:.2f}%, annualized return volatility {ann_vol[t]*100:.2f}%, and correlation {return_corr.loc[t].sum():.2f}')
4     f'annualized return volatility {ann_vol[t]*100:.2f}%, and correlation {return_corr.loc[t].sum():.2f}'
5     f'{((return_corr.loc[t].sum()-1)/(len(return_corr)-1)):.2f}'
```

QUANTITATIVE ANCHORS

SPY (45%): annual return 20.02%, Sharpe 1.147, annualized return volatility 18.50%, correlation 0.999
 AAPL (5%): annual return 35.01%, Sharpe 1.152, annualized return volatility 20.00%, correlation 0.999
 JNJ (20%): annual return 57.99%, Sharpe 2.277, annualized return volatility 15.00%, correlation 0.999
 XOM (15%): annual return 49.38%, Sharpe 1.529, annualized return volatility 15.00%, correlation 0.999
 EFA (15%): annual return 20.87%, Sharpe 1.010, annualized return volatility 15.00%, correlation 0.999

```
1 # This allocation is 100% in equity/ETF positions, so portfolio for David is 100% in equity/ETF positions
2 initial_value = 40000
3 stress_loss_pct = 0.20
4 stress_value = initial_value * (1-stress_loss_pct)
5 stress_loss_dollars = initial_value - stress_value
6
7 print(f'Initial portfolio = ${initial_value:,.2f}')
8 print(f'20% stress loss = ${initial_value:,.2f} × 0.20 = ${stress_value:,.2f}')
9 print(f'Ending value = ${initial_value:,.2f} - ${stress_loss_dollars:,.2f} = ${stress_value:,.2f}')
10 print(f'Loss percentage = 20.00%, which is below David\'s 25% maximum')

```

Initial portfolio = \$40,000.00
 20% stress loss = \$40,000.00 × 0.20 = \$8,000.00
 Ending value = \$40,000.00 - \$8,000.00 = \$32,000.00
 Loss percentage = 20.00%, which is below David's 25% maximum.

```
1 # Section 5 – Additional Analysis
2 # Additional Analysis 1: Maximum drawdown
3 wealth_index = (1 + daily_returns).cumprod()
4 running_peak = wealth_index.cummax()
5 drawdown = wealth_index / running_peak - 1
6 max_drawdown = drawdown.min().sort_values()
7
8 print('ADDITIONAL ANALYSIS 1 – MAXIMUM DRAWDOWN')
9 display((max_drawdown*100).round(2).to_frame('Max Drawdown %'))
10
11 plt.figure(figsize=(14,7))

```



```

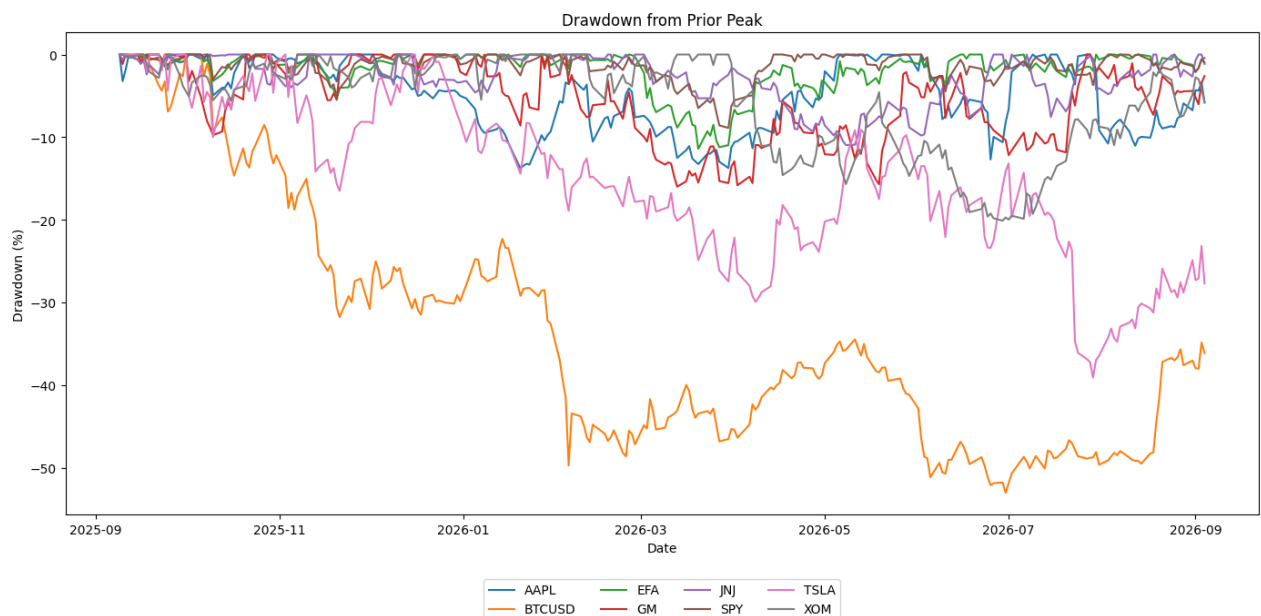
12 for t in drawdown.columns:
13     plt.plot(drawdown.index, drawdown[t]*100, label=t)
14 plt.title('Drawdown from Prior Peak')
15 plt.xlabel('Date')
16 plt.ylabel('Drawdown (%)')
17 plt.legend(loc='upper center', bbox_to_anchor=(0.5,-0.12), ncol=4)
18 plt.tight_layout()
19 plt.savefig(f'{RESULTS_DIR}/max_drawdown.png', dpi=200, bbox_inches='tight')
20 plt.show()

```

ADDITIONAL ANALYSIS 1 — MAXIMUM DRAWDOWN

Max Drawdown % 

Ticker	
BTCUSD	-53.06
TSLA	-39.10
XOM	-20.11
GM	-16.00
AAPL	-13.80
EFA	-11.42
JNJ	-10.96
SPY	-8.88



```
1 # Additional Analysis 2: Sortino Ratio and Downside Risk
2
3 # Daily risk-free rate specified in the assignment
4 rf_daily = 0.045 / 252
5
6 # Downside returns: only observations below the daily risk-free rate
7 downside_returns = daily_returns - rf_daily
8 downside_returns = downside_returns.where(downside_returns < 0, 0)
9
10 # Annualized downside deviation
11 downside_deviation = downside_returns.std() * np.sqrt(252)
12
13 # Annualized excess return
14 annualized_excess_return = (daily_returns.mean() - rf_daily) * 252
15
16 # Sortino Ratio
17 sortino_ratio = annualized_excess_return / downside_deviation
18
19 # Create summary table
20 sortino_table = pd.DataFrame({
21     'Annual Return %': annual_return_pct,
22     'Annualized Volatility %': ann_vol * 100,
23     'Downside Deviation %': downside_deviation * 100,
24     'Sharpe Ratio': sharpe,
25     'Sortino Ratio': sortino_ratio
26 })
27
```

```
28 sortino_table = sortino_table.sort_values(
29     'Sortino Ratio',
30     ascending=False
31 ).round(3)
32
33 print("ADDITIONAL ANALYSIS 2 – DOWNSIDE RISK AND SORTINO RATIO")
34 display(sortino_table)
35
36 # Bar chart of Sortino ratios
37 plt.figure(figsize=(10,6))
38
39 sortino_ratio.sort_values(ascending=False).plot(
40     kind='bar'
41 )
42
43 plt.axhline(0, linewidth=1)
44 plt.title('Sortino Ratio by Asset')
45 plt.xlabel('Asset')
46 plt.ylabel('Sortino Ratio')
47 plt.xticks(rotation=45)
48 plt.tight_layout()
49
50 plt.savefig(
51     f'{RESULTS_DIR}/sortino_ratio.png',
52     dpi=200,
53     bbox_inches='tight'
54 )
55
56 plt.show()
57
58 # Automatically identify best and worst
59 best_sortino = sortino_ratio.idxmax()
60 worst_sortino = sortino_ratio.idxmin()
61
62 print(
63     f'Highest Sortino Ratio: {best_sortino} '
64     f'({sortino_ratio[best_sortino]:.3f})'
65 )
66
67 print(
68     f'Lowest Sortino Ratio: {worst_sortino} '
69     f'({sortino_ratio[worst_sortino]:.3f})'
70 )
71
72 print("\nDownside deviation ranking:")
```

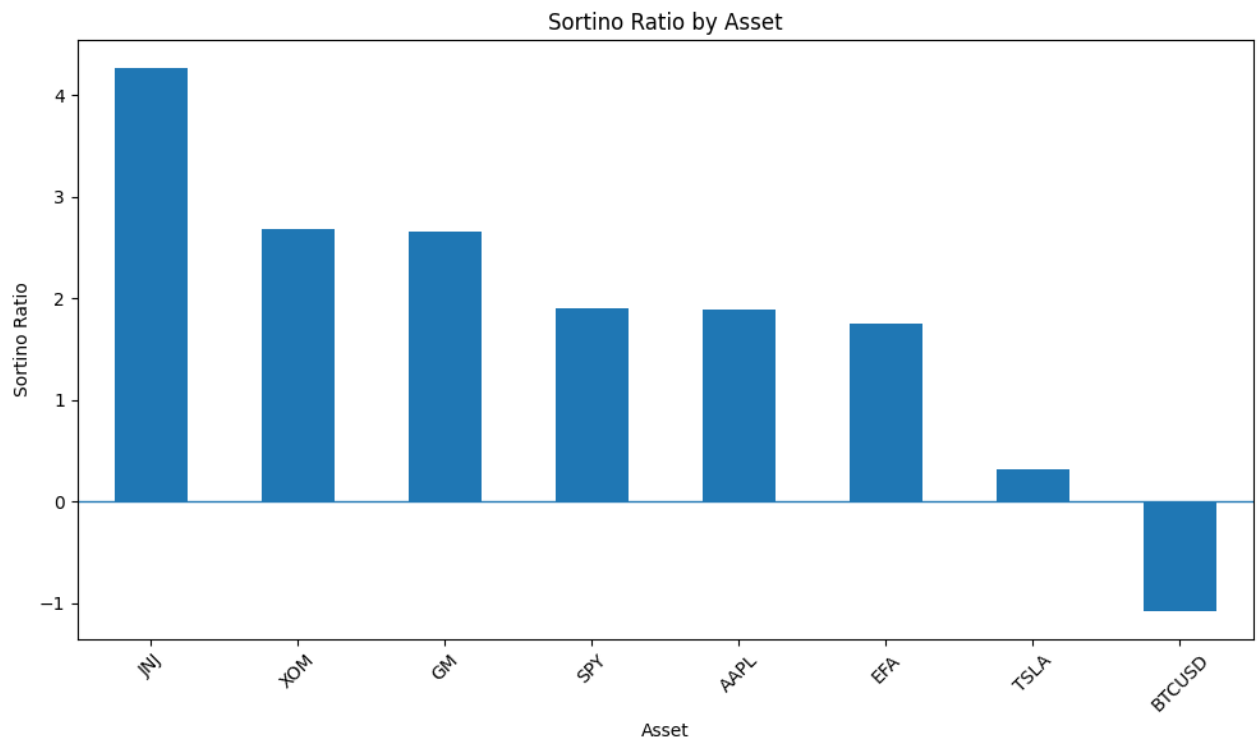
```

73 display(
74 ....(downside_deviation*100)
75     .sort_values()
76     .round(2)
77     .to_frame('Annualized Downside Deviation %')
78 )

```

ADDITIONAL ANALYSIS 2 — DOWNSIDE RISK AND SORTINO RATIO

	Annual Return %	Annualized Volatility %	Downside Deviation %	Sharpe Ratio	Sortino Ratio
Ticker					
JNJ	57.99	19.083	10.170	2.277	4.273
XOM	49.38	25.691	14.676	1.529	2.676
GM	52.09	34.194	16.392	1.272	2.653
SPY	20.02	12.833	7.734	1.147	1.904
AAPL	35.01	25.122	15.333	1.152	1.887
EFA	20.87	15.672	9.053	1.010	1.749
TSLA	2.22	47.470	28.699	0.190	0.314
BTCUSD	-28.91	43.962	26.889	-0.664	-1.086



Highest Sortino Ratio: JNJ (4.273)

Lowest Sortino Ratio: BTCUSD (-1.086)

LOWEST DOWNSIDE RISK: SPY (7.73%),

Downside deviation ranking:

Annualized Downside Deviation %	
Ticker	
SPY	7.73
EFA	9.05
JNJ	10.17
XOM	14.68
AAPL	15.33
GM	16.39
BTCUSD	26.89
TSLA	28.70

Next steps: [New interactive sheet](#)